

Chapter 4 Writing Classes

CHAPTER OBJECTIVES

- Explore techniques for identifying the classes and objects needed in a program.
- Discuss the structure and content of a class definition.
- Establish the concept of object state using instance data.
- Describe the effect of visibility modifiers on methods and data.
- Explore the structure of a method definition, including parameters and return values.
- Discuss the structure and purpose of a constructor.
- Discuss the relationships among classes.
- Describe the effect of the `static` modifier on methods and data.
- Discuss issues related to the design of methods, including method decomposition and method overloading.

In the chapters leading up to this one, we made use of existing classes and objects for the services they offered, and we worked through a range of fundamental programming constructs. Building on that foundation, we are now prepared to take a significant step forward: designing our own classes from scratch, which is the very heart of object-oriented programming. This chapter examines the essentials of class definitions — including how methods are structured, how data is scoped and encapsulated, how static class members work, and how method overloading is achieved.

4.1 Classes and Objects Revisited

Chapter 1 introduced the foundational concepts of object-oriented programming, offering a brief overview of objects and classes. In previous Chapters we put several predefined classes from the Java standard library to practical use, creating objects and calling on them to perform specific tasks.

In this chapter we shift our focus to writing our own classes. While the existing class libraries supply a rich collection of useful building blocks, the true substance of object-oriented development lies in designing and implementing the classes we need ourselves, tailored to the specific requirements of our programs.

Recall the central relationship between a class and an object: a class is a blueprint, and any object created from it is a concrete realization of that blueprint. The `String` class represents the abstract concept of a character string, while each individual `String` object holds a specific sequence of characters.

Consider a further example. Suppose we define a class called `Student` to represent a student at a university. An object created from that class would correspond to one particular student. The class captures the general notion of what a student is, while each object instantiated from it represents an actual, specific individual. A system built to support

university administration might have a single **Student** class and thousands of **Student** objects, one for each enrolled student.

Recall also that every object has a state — defined by the current values of its attributes. For a student, these attributes might include name, mailing address, declared major, and cumulative grade point average. The **Student** class establishes that all students possess these attributes; each **Student** object then stores the particular values associated with one individual. In Java, an object's attributes are represented by variables declared within the class definition.

An object also has behaviors — the set of operations it can perform. For a student, those behaviors might include updating a mailing address or computing the current grade point average. The **Student** class defines how those operations work in detail, and those operations can then be invoked on any particular **Student** object. Behaviors may also modify an object's state. In Java, an object's behaviors are implemented as methods declared within the class.

Identifying Classes and Objects

One of the most fundamental challenges in object-oriented software design is determining which classes to create. The classes we define shape the objects we will work with throughout the system, so careful thought at this stage pays dividends throughout development.

KEY CONCEPT

The nouns in a problem description may indicate some of the classes and objects needed in a program.

A practical starting point for identifying candidate classes is to look at the nouns in the program requirements or functional specification. Objects in a problem description tend to be expressed as nouns. Literally scanning the text for nouns and circling them is a useful exercise for surfacing potential classes and objects.

Of course, not every noun in a requirements document will map directly to a class. Some nouns may be better represented as attributes of other objects, and others may not need explicit representation in the program at all. This exercise is simply an entry point — a technique for beginning to think about the types of objects the program will need to manage.

It is also worth keeping in mind that a class represents a group of objects sharing common characteristics. A plural noun in the specification, such as "products," often signals the need for a class representing a single instance of that thing — in this case, a **Product** class. Even when only one instance of a particular kind of object is needed, defining it as a class is often the right choice.

Class names that represent objects should generally be singular nouns — `Coin`, `Student`, `Message` — reflecting the fact that each class defines one kind of thing from which any number of instances can be created.

Another decision that arises repeatedly is whether a particular concept should be modeled as its own class or simply as a primitive attribute of another class. A developer might initially represent an employee's salary as a plain integer, and that may suffice for straightforward processing. But on reflection, the salary might turn out to be tied to a pay grade with its own upper and lower bounds and associated business logic — in which case, encapsulating all of that data and behavior in a dedicated `Salary` class becomes the more appropriate design.

Striking the right level of granularity is important. An overly fine-grained design that creates a separate class for every minor variation can introduce unnecessary complexity. An overly coarse design may lump things together in ways that make the code harder to understand and extend. The right balance depends entirely on what the software is meant to accomplish.

Beyond classes that represent objects from the problem domain, a program will often need additional supporting classes. For example, alongside a `Member` class, a separate class for managing the full collection of club members may prove necessary.

Finally, during design it is always worth checking whether any of the identified classes already exist — in the Java standard library, in code carried over from previous projects, or in a third-party library. Reusing existing, proven code rather than writing everything from scratch is one of the most tangible benefits of object-oriented development.

Assigning Responsibilities

Alongside identifying the classes a program needs, the design process involves assigning responsibilities to each class. Every class defines an object with specific behaviors, expressed as methods. Every activity the program must carry out must be assigned to the methods of some class — each class takes ownership of certain tasks as part of the overall design.

Since class behaviors describe actions, their names are typically drawn from verbs. The methods of a class are the verbs to the class's noun.

Deciding which class should own a particular responsibility is not always straightforward. A thoughtful designer considers multiple possible assignments before committing, and sometimes that analysis surfaces the need for an entirely new class to handle a responsibility cleanly.

In the early stages of design, it is not necessary to enumerate every method a class will eventually contain. Identifying the primary responsibilities of each class and thinking through how those responsibilities will translate into concrete methods is typically a sufficient starting point.

4.2 Anatomy of a Class

With these conceptual foundations in place, let's turn to the programming details. In all of the examples we have written up to this point, each program consisted of a single class with a single `main` method. These classes often instantiated objects from predefined library classes and used them for the services they provided. The internal workings of those library classes were not our concern — we only needed to understand how to interact with them and trust that they delivered on their contracts.

KEY CONCEPT

The heart of object-oriented programming is defining classes that represent objects with well-defined state and behavior.

We will continue to rely on library classes, but from here on we will also be designing and implementing our own. As a first example, the `SnakeEyes` class in Listing 4.1 contains a `main` method that creates two `Die` objects and rolls them repeatedly, counting how often both show a `1` on the same throw — the result known as snake eyes.

The critical difference between this example and those in earlier chapters is that `Die` is not part of the Java standard library. For this program to compile and run, we must write the `Die` class ourselves, defining precisely the services we want `Die` objects to provide.

A class can contain two kinds of members: data declarations and method declarations. The data declarations define the information that each object of that class will store. The method declarations define the operations those objects can perform. Classes we have written previously follow this same model, though they contained no class-level data and only a single `main` method.

Genuine object-oriented programming, however, centers on defining classes that represent objects with meaningful, well-encapsulated state and behavior. A `Die` object, for example, is at any moment showing a particular face value — that is its state. It also supports operations such as rolling itself or returning its current face value — those are its behaviors.

The `Die` class is shown in Listing 4.2. It defines two pieces of data: an integer constant `MAX` representing the highest possible face value, and an integer variable `faceValue` holding the face currently shown. It also defines a constructor named `Die` and four regular methods: `roll`, `setFaceValue`, `getFaceValue`, and `toString`.

As we covered in previous Chapters, a constructor is a special method that shares the name of the class. The `Die` constructor is invoked automatically whenever the `new` operator is used to create a new `Die` instance, as happens twice in the `SnakeEyes` `main` method. The remaining methods define the various services `Die` objects expose.

Each method in the class is preceded by a block of documentation that explains its purpose. This is not merely a good habit — it is essential for anyone who needs to understand or maintain the code, and it also provides a clean visual structure that makes it easy to locate and read individual methods.

From this perspective, the `Die` class looks no different from any class we have used before. The only meaningful distinction is that we wrote it ourselves rather than finding it in the standard library.

The `roll` method generates a new random face value and returns it to the caller. The `getFaceValue` method returns the current face value without modifying it. The `setFaceValue` method allows the face value to be set explicitly to any valid value — analogous to reaching over and manually turning a physical die. The `toString` method returns a string representation of the die's current state. We will examine the details of each method definition as we work through the rest of this chapter.

The `toString` method deserves particular mention. It is called automatically whenever a `println` or `print` method receives an object as an argument, and also whenever an object is concatenated to a string. Every object has a default `toString` inherited from the root of the Java class hierarchy, but the default output is rarely meaningful. Defining a meaningful `toString` in every class you create is therefore strongly recommended. Inheritance — the mechanism that makes the default `toString` available.

In the examples throughout this book, each class is stored in its own separate file. Java does permit multiple classes to be defined within a single file, but with a restriction: only one class in the file may be declared `public`, and the filename must match the name of that public class exactly. The `Die` class, for instance, lives in a file named `Die.java`.

Instance Data

In the `Die` class, both `MAX` and `faceValue` are declared inside the class body but outside any method. The location of a variable's declaration determines its scope — the region of code from which it can be referenced.

KEY CONCEPT

The scope of a variable, which determines where it can be referenced, depends on where it is declared.

By being declared at the class level rather than inside any individual method, `MAX` and `faceValue` are accessible from any method within the class.

The variable `faceValue` is an example of instance data. Each time a new instance of the `Die` class is created, a separate, independent copy of `faceValue` is allocated in memory for that object. This is what allows each `Die` object to maintain its own distinct state. One die

could be showing a 5 while another shows a 2 simultaneously — possible only because each object holds its own copy of `faceValue`.

We can represent this situation as follows:

die1 → faceValue: 5

die2 → faceValue: 2

The reference variables `die1` and `die2` each point to a distinct `Die` object in memory. Those objects are independent of one another, each holding its own `faceValue` with its own storage.

Java automatically initializes class-level variables to default values — numeric types are initialized to zero, for example. Despite this, explicitly initializing variables (typically within a constructor) is good practice. It makes the intended initial state clear to anyone reading the code, rather than leaving them to rely on knowledge of the language's default initialization rules.

UML Class Diagrams

As programs grow in complexity and involve multiple interacting classes, it becomes increasingly valuable to have a graphical notation for capturing and communicating the design. Throughout the remainder of this book we use UML diagrams for this purpose. UML — the Unified Modeling Language — has become the industry-standard notation for representing object-oriented program designs.

KEY CONCEPT

A UML class diagram helps us visualize the contents of and relationships among the classes of a program.

Several categories of UML diagrams exist, each tailored to represent different aspects of a system. Our primary focus is on UML class diagrams, which show what each class contains and how classes relate to one another.

In a UML class diagram, each class is drawn as a rectangle, optionally divided into three sections: the class name, its attributes (data), and its operations (methods). Figure 5.5 shows the class diagram for the `SnakeEyes` program.

UML is designed to be language-independent, so its notation does not always mirror Java syntax exactly. For example, in UML, a variable's type is written after the variable name, separated by a colon, and method return types are shown the same way. Initial values for attributes can optionally be included in the diagram, as shown for the `MAX` constant in Figure 5.5. The `+` and `-` symbols preceding member names indicate their visibility, which is covered in the next section.

Connecting lines between classes in a UML diagram, called associations, indicate that a relationship of some kind exists between them — typically that one class creates or uses objects of the other, or that one class calls methods of the other. Associations can be labeled to describe the nature of the relationship.

A directed association — drawn with an arrowhead — signals that the relationship runs primarily in one direction. The arrow from `SnakeEyes` to `Die` in Figure 5.5 indicates that `SnakeEyes` knows about and uses `Die`, but the reverse is not true.

Multiplicity annotations at the ends of an association line indicate how many objects of each class participate in the relationship. In Figure 5.5, the annotation indicates that `SnakeEyes` is associated with exactly two `Die` objects. Both ends of an association can carry multiplicity values, and ranges or wildcards can be used when the exact number is variable or unknown — as we will see in later diagrams.

Different kinds of relationships between classes are expressed using different types of lines and arrowheads, which we will introduce as the corresponding programming concepts are presented throughout the book.

UML diagrams are highly flexible. A diagram can include as much or as little detail as is useful for communicating a particular design decision. We might omit the data and method sections of a class entirely when those details are irrelevant for the point being illustrated. For instance, Figure 5.5 notes that `Die` makes use of the `Math` class, but does not spell out `Math`'s internal details. The `String` class could similarly have been shown, but it is used so pervasively in Java programs that including it explicitly in every diagram would add clutter without adding insight.

```
// LISTING 4.1

//*****

// SnakeEyes.java

//

// Demonstrates the use of a programmer-defined class.

//*****

public class SnakeEyes
{

    //-----

    // Creates two Die objects and rolls them several times, counting
```

```

// the number of snake eyes that occur.

//-----

public static void main(String[] args)
{
    final int ROLLS = 500;

    int num1, num2, count = 0;

    Die die1 = new Die();

    Die die2 = new Die();

    for (int roll = 1; roll <= ROLLS ; roll++)
    {
        num1 = die1.roll();

        num2 = die2.roll();

        if (num1 == 1 && num2 == 1) //check for snake eyes
            count++;
    }

    System.out.println("Number of rolls: " + ROLLS);

    System.out.println("Number of snake eyes: " + count);

    System.out.println("Ratio: " + (float) count / ROLLS);
}
}

```

OUTPUT

Number of rolls: 500

Number of snake eyes: 12

Ratio: 0.024

```
// LISTING 4.2

//*****

// Die.java

//

// Represents one die (singular of dice) with faces showing values
// between 1 and 6.

//*****

public class Die
{
    private final int MAX = 6; // maximum face value
    private int faceValue; // current value showing on the die

    //-----
    // Constructor: Sets the initial face value of this die.
    //-----

    public Die()
    {
        faceValue = 1;
    }

    //-----
    // Computes a new face value for this die and returns the result.
    //-----
}
```

```
public int roll()

{

    faceValue = (int) (Math.random() * 6) + 1;

    return faceValue;

}


//-----

// Face value mutator. The face value is not modified if the
// specified value is not valid.

//-----

public void setFaceValue(int value)

{

    if (value > 0 && value <= MAX)

        faceValue = value;

}


//-----

// Face value accessor.

//-----


public int getFaceValue()

{

    return faceValue;

}


//-----

// Returns a string representation of this die.
```

```
//-----  
  
public String toString()  
  
{  
  
    String result = Integer.toString(faceValue);  
  
    return result;  
  
}  
  
}
```

4.3 Encapsulation

In Chapter 1 we established that an object should be self-governing — meaning that its internal data should be modifiable only through that object's own methods. The methods of the `Die` class, for example, should be the sole authority over changes to `faceValue`. Code outside the class should be prevented from reaching in and altering that variable directly. This principle is called encapsulation.

A properly encapsulated object interacts with the rest of the system exclusively through the specific set of methods that define its services. These methods constitute the interface between the object and any external code that makes use of it.

Figure 5.6 depicts this relationship graphically. The code that uses an object — commonly referred to as the client — should never access the object's variables directly. Instead, the client calls the object's methods, and those methods are the only pathway through which the encapsulated data can be read or modified. The `main` method in `SnakeEyes`, for instance, calls the `roll` method on each die object. It cannot — and should not — access `faceValue` directly.

In Java, encapsulation is enforced through the use of modifiers. A modifier is a reserved word that specifies a particular characteristic of a language construct. We encountered the `final` modifier in Chapter 2 as a means of declaring constants. Java provides several modifiers that apply in various contexts; some can be combined, while certain combinations are invalid. The full set of Java modifiers is summarized in Appendix E, and we introduce them at appropriate points throughout the book.

KEY CONCEPT

An object should be encapsulated, guarding its data from inappropriate access.

Visibility Modifiers

A subset of Java's modifiers are called visibility modifiers because they govern whether the members of a class can be accessed from outside that class. The reserved words `public` and `private` are visibility modifiers that can be applied to both variables and methods. A member declared with `public` visibility can be accessed directly from code outside the object. A member declared with `private` visibility is accessible only from within the class itself — nothing outside the class can reference it. A third visibility level, `protected`, is relevant only in the context of inheritance.

Declaring instance variables as `public` violates encapsulation by allowing external code to read and modify the data freely. Instance data should therefore always be declared with `private` visibility. Only the methods of the class itself can then access or modify that data.

The appropriate visibility for a method depends on its intended purpose. Methods that deliver services to external clients must be declared `public` so that callers can invoke them. These are often called service methods. Methods that exist purely to assist other methods within the class have no reason to be visible externally and should be declared `private`. These internal helpers are sometimes called support methods.

KEY CONCEPT

Instance variables should be declared with `private` visibility to promote encapsulation.

One nuance worth noting: constants declared with `final` can reasonably be given `public` visibility. Because their values cannot be altered once assigned, the core concern of encapsulation — preventing unauthorized modification of data — does not apply to them.

In UML class diagrams, visibility is conveyed with a symbol preceding each member name: a plus sign (+) for `public` and a minus sign (-) for `private`.

Accessors and Mutators

Because instance data is typically declared `private`, a class must provide methods that allow clients to read and update those values in a controlled way. A method like `getFaceValue` in the `Die` class is called an accessor method — it provides read-only access to a specific value without exposing the underlying variable. A method like `setFaceValue` is called a mutator method — it allows a value to be changed through a defined, validated pathway.

By convention, accessor method names take the form `getX` and mutator method names take the form `setX`, where `X` identifies the value being accessed or modified. These methods are commonly referred to informally as getters and setters.

For example, a class that contains an instance variable `height` would typically also provide `getHeight` and `setHeight` methods. Note that this convention capitalizes the first letter of the variable name when it appears in the method name, which is consistent with the broader Java convention for method naming.

KEY CONCEPT

Most objects contain accessor and mutator methods to allow the client to manage data in a controlled manner.

Some methods provide accessor or mutator behavior as a secondary consequence of their primary function. The `roll` method in the `Die` class, for instance, both changes `faceValue` and returns the new value. And the `roll` method is guaranteed to keep the face value within the valid range. Similarly, the `setFaceValue` method validates the incoming value before applying it. Well-designed service methods must enforce these kinds of constraints — encapsulation gives the object the authority to do so.

The program in Listing 4.3 illustrates these principles with a `Coin` object. It creates one coin, flips it one thousand times, and counts how many times heads and tails appear. The `isHeads` method is used as the condition of an `if` statement to determine the outcome of each flip.

KEY CONCEPT

The way a class represents an object's state should be independent of how that object is used.

The `Coin` class in Listing 4.4 stores an integer constant `HEADS` representing the heads face value, and an integer instance variable `face` that records the current state of the coin — either `0` for heads or `1` for tails. The constructor kicks off the coin's existence by immediately calling `flip`, which randomly assigns a face value. The `isHeads` method returns a `boolean` based on the current value of `face`, and `toString` returns a human-readable string describing the coin's current state.

The internal representation of the coin's state is a deliberate design choice. We could have used a `boolean`, a character string, or an enumerated type just as effectively. An integer was chosen because `Math.random` returns a numeric value, which made the state assignment straightforward without requiring additional type conversions.

From the client's perspective — that is, from the `CountFlips` program's point of view — the internal representation is completely irrelevant. The client only interacts with the `Coin` object through its publicly defined methods. The internal details of the class could be restructured entirely, and as long as the `isHeads` method continued to behave correctly, the client code would not need to change at all.

It is also worth observing that the `Coin` class deliberately omits classic getter and setter methods. The only mechanism for changing the coin's state is the `flip` method, which randomizes it. Unlike a `Die` object — whose face value can be set explicitly via

`setFaceValue` — a coin's state cannot be directly dictated by the caller. This is an intentional design decision that reflects the real-world behavior of a coin, though it could reasonably be designed differently depending on the requirements.

The `FlipRace` class in Listing 5.5 demonstrates another use of the `Coin` class. It creates two `Coin` objects and flips them in parallel until one of them lands on heads three times consecutively.

In the output of `FlipRace`, notice that `coin1` and `coin2` are concatenated directly into strings inside `println`. As noted earlier, this causes each object's `toString` method to be invoked automatically, returning the string representation without any explicit call to `toString`.

After each flip, the conditional operator updates each coin's consecutive-heads counter: if the result is heads, the counter increments; otherwise it resets to zero. The `while` loop runs until at least one counter reaches the target of three. An `if-else` chain then determines and announces the winner — or declares a tie if both reached the goal on the same iteration.

`FlipRace` reuses the `Coin` class just as `CountFlips` did, demonstrating one of the most tangible benefits of sound object-oriented design: a well-written class can serve multiple programs without any modification.

```
// LISTING 4.3
//*****
// CountFlips.java
//
// Demonstrates the use of programmer-defined class.
//*****
public class CountFlips
{
    //-----
    // Flips a coin multiple times and counts the number of heads
    // and tails that result.
    //-----
    public static void main(String[] args)
    {
        final int FLIPS = 1000;
        int heads = 0, tails = 0;

        Coin myCoin = new Coin();

        for (int count = 1; count <= FLIPS ; count++)
        {
            myCoin.flip();
            if (myCoin.isHeads())
```

```

        heads++;
    else
        tails++;
    }

    System.out.println("Number of flips: " + FLIPS);
    System.out.println("Number of heads: " + heads);
    System.out.println("Number of tails: " + tails);
}
}

```

OUTPUT

Number of flips: 1000
 Number of heads: 486
 Number of tails: 514

```

// LISTING 4.4

//*****

// Coin.java
//
// Represents a coin with two sides that can be flipped.
//*****

public class Coin
{

    private final int HEADS = 0; // tails is 1

    private int face;           // current side showing

    //-----

    // Sets up this coin by flipping it initially.

    //-----

    public Coin ()

    {

```

```
        flip();

    }

    //-----
    // Flips this coin by randomly choosing a face value.
    //-----

    public void flip()

    {

        face = (int) (Math.random() * 2);

    }

    //-----
    // Returns true if the current face of this coin is heads.
    //-----

    public boolean isHeads ()

    {

        return (face == HEADS);

    }

    //-----
    // Returns the current face of this coin as a string.
    //-----

    public String toString()

    {

        return (face == HEADS) ? "Heads" : "Tails";

    }

}
```



```
}
```

```
// LISTING 4.5

//*****

// FlipRace.java

//

// Demonstrates the reuse of programmer-defined class.

//*****

public class FlipRace
{
    //-----

    // Flips two coins until one of them comes up heads three times
    // in a row.

    //-----

    public static void main(String[] args)
    {
        final int GOAL = 3;

        int count1 = 0, count2 = 0;

        Coin coin1 = new Coin(), coin2 = new Coin();

        while (count1 < GOAL && count2 < GOAL)
        {
            coin1.flip();

            coin2.flip();

            System.out.println("Coin 1: " + coin1 + "\tCoin 2: " +
coin2);

```

```

        // Increment or reset the counters

        count1 = (coin1.isHeads()) ? count1+1 : 0;

        count2 = (coin2.isHeads()) ? count2+1 : 0;

    }

    if (count1 < GOAL)

        System.out.println("Coin 2 Wins!");

    else

        if (count2 < GOAL)

            System.out.println("Coin 1 Wins!");

        else

            System.out.println("It's a TIE!");

    }
}

```

4.4 Anatomy of a Method

We have established that a class is composed of data declarations and method declarations. Let's now examine method declarations in greater depth.

As stated in Chapter 1, a method is a named group of programming statements. A method declaration defines the code that executes whenever that method is invoked. In Java, every method belongs to a specific class.

A method's header specifies the return type, the method name, and the list of parameters the method accepts. The statements that form the method's body are enclosed in a block delimited by braces. The `main` method we have written in every program example follows the same syntactic structure as any other method.

When a method is called, the flow of control transfers to that method's body. Its statements execute one by one, and when the method finishes, control returns to the point in the calling code immediately following the invocation.

If the method being called belongs to the same class as the calling method, only the method name is needed to invoke it. If it belongs to a different class, it is invoked through a reference to an object of that class — a pattern we have used extensively.

The `Transactions` class in Listing 4.6 provides a further example. Its `main` method creates several `Account` objects and calls a variety of their methods. Programs whose sole purpose is to exercise and test the functionality of other classes are called driver programs. They are a common and practical tool during development and testing.

The `Account` class in Listing 4.7 represents a basic bank account. It holds instance data for the account owner's name, the account number, and the current balance, along with a constant for the interest rate. The constructor accepts three parameters that are used to initialize the instance data at the time the object is created. The remaining methods implement the core account operations — depositing funds, withdrawing funds, and applying interest. Each method validates its inputs before making any changes: the `withdraw` method, for example, rejects any withdrawal of a negative amount (which would otherwise function as an unauthorized deposit) and also ensures the balance is sufficient. Because all legitimate modifications to the balance flow through these controlled methods, there is no need for a generic `setBalance` mutator.

The initial state of the three `Account` objects created in `Transactions` can be depicted as follows:

acct1 → name: "Ted Murphy" acctNumber: 72354 balance: 25.59

acct2 → name: "Angelica Adams" acctNumber: 69713 balance: 500.00

acct3 → name: "Edward Demsey" acctNumber: 93757 balance: 769.32

The `return` Statement

The return type declared in a method header may be a primitive type, a class name, or the reserved word `void`. When a method produces no return value, `void` is used — as is always the case with the `main` method. The `setFaceValue` method in `Die` and the `flip` method in `Coin` are also `void`.

The `getFaceValue` and `roll` methods in `Die` each return an `int`. The `isHeads` method in `Coin` returns a `boolean`. Several methods in `Account` return a `double` representing the updated balance after a transaction. The `toString` methods in each class return a `String` object.

KEY CONCEPT

The value returned from a method must be consistent with the return type specified in the method header.

Any method that declares a non-`void` return type must contain a `return` statement. When a `return` statement is executed, control transfers immediately back to the calling method and processing resumes from there. A `return` statement consists of the reserved word `return` followed by an expression whose type must be compatible with the declared return type.

A `void` method generally does not need a `return` statement — it returns to the caller automatically when the end of the method body is reached. Such methods may, however, contain a bare `return` statement (without an expression) to exit early if needed.

Using multiple `return` statements within a single method is permitted but generally not recommended. The preferred style is a single `return` statement as the final line of the method body, unless consolidating to a single return point would make the code unnecessarily convoluted.

The return value of a method can be silently discarded by the caller. In the `Transactions` program, for example:

```
acct1.deposit(44.10);
```

This invocation executes the `deposit` method normally, updating the balance as expected, but the calling code simply ignores the balance value that was returned.

Constructors have no return type — not even `void` — and therefore cannot return a value. Constructors are discussed further in the section below.

Parameters

We introduced the concept of parameters in Chapter 2, defining them as values passed into a method at the point of invocation. Parameters supply the data a method needs to carry out its work. Let's now examine this mechanism more carefully.

The method declaration specifies precisely how many parameters the method accepts and what type each one must be. More specifically, the parameter list in the method header declares the type and name of each expected value. The corresponding parameter list at the call site provides the actual values to be passed for that particular invocation.

Parameters named in the method header are called formal parameters. The values supplied at the call site are called actual parameters, or arguments. Both lists are enclosed in parentheses immediately after the method name. When there are no parameters, an empty pair of parentheses is used.

Among the classes we have seen, most methods take no parameters. The `setFaceValue` method of `Die` is an exception — it accepts a single integer specifying the new face value. The `Account` constructor accepts multiple parameters of different types to supply the initial

values for the new object's instance data. The `withdraw` method in `Account` accepts two `double` parameters; note that even when multiple parameters share the same type, each must list its type individually in the header.

KEY CONCEPT

When a method is called, the actual parameters are copied into the formal parameters.

Formal parameters behave as local variables within the method. They come into existence when the method is called, initialized with copies of the corresponding actual parameter values, and they cease to exist when the method returns. Actual parameters may be literals, variables, or fully evaluated expressions — in the last case, the expression is computed before the call and the result is passed in.

The correspondence between actual and formal parameters is positional: the first actual parameter is copied into the first formal parameter, the second into the second, and so on. The types of actual parameters must be compatible with — though not necessarily identical to — the types of their corresponding formal parameters.

In the `Transactions` program, the following call is made:

```
acct2.withdraw(480, 1.50);
```

The first argument, `480`, is an integer, while the `withdraw` method declares its first parameter as `double`. This is valid because widening conversions are permitted in parameter passing, just as they are in assignment — an `int` can always be assigned to a `double` without loss of information.

Local Data

The scope of a variable or constant is the portion of the program in which it can be validly referenced. Variables can be declared inside a method, making them local data rather than instance data. Instance data is declared at the class level, outside any method; local data is declared within a specific method.

Local data has scope limited entirely to the method in which it is declared. The variable `result` declared in the `toString` method of the `Die` class, for example, is local to that method. Any reference to `result` from any other method in `Die` would cause a compiler error. A local variable simply does not exist anywhere outside its declaring method. Instance data, by contrast, is accessible from any method in the class.

Because local and instance variables can coexist at different scopes, it is technically possible to declare a local variable with the same name as an instance variable. Within that method, the name resolves to the local variable, effectively shadowing the instance variable. This is a potential source of confusion and should be avoided.

The formal parameters in a method header also behave as local data. They are created when the method is invoked, hold the copied values of the actual parameters during execution, and are destroyed when the method exits. In the `Account` constructor, for instance, the formal parameter `owner` exists only for the duration of the constructor call — to persist its value beyond that, it is explicitly assigned to the `name` instance variable.

KEY CONCEPT

A variable declared in a method is local to that method and cannot be used outside of it.

Constructors Revisited

KEY CONCEPT

A constructor cannot have any return type, even `void`.

When a class is defined, a constructor is almost always provided to handle the initialization of newly created objects. Constructors establish the initial state of an object, typically by assigning values to instance variables.

A constructor differs from a regular method in two fundamental ways. First, a constructor's name must be identical to the name of the class — `Die` for the `Die` class, `Account` for the `Account` class, and so on. Second, a constructor has no return type at all, not even `void`.

A common mistake is to write `void` in front of a constructor. From the compiler's perspective, adding any return type — including `void` — transforms the declaration into an ordinary method that happens to share the class name. The method can no longer be used as a constructor, and the resulting compiler error messages are often cryptic and difficult to trace.

The appropriate way to use a constructor depends on the design of the class. The `Die` constructor sets the initial face value to `1`. The `Coin` constructor calls `flip` to put the coin in a random initial state. The `Account` constructor accepts parameters and uses them to initialize all three instance variables at once — a common pattern for constructors that need externally provided values to set up a meaningful initial state.

If a class definition includes no constructor at all, Java automatically generates a default constructor that takes no parameters and performs no initialization. However, once the programmer provides at least one constructor — with or without parameters — the automatically generated default constructor is no longer created.

```
// LISTING 4.6
//*****
// Transactions.java
```

```
//
// Demonstrates the creation and use of multiple Account objects.
//*****
public class Transactions
{
    //-----

    // Creates some bank accounts and requests various services.

    //-----

    public static void main(String[] args)
    {
        Account acct1 = new Account ("Ted Murphy", 72345, 25.59);
        Account acct2 = new Account ("Angelica Adams", 69713, 500.00);
        Account acct3 = new Account ("Edward Demsey", 93757, 769.32);

        acct1.deposit(44.10); // return value ignored

        double adamsBalance = acct2.deposit(75.25);

        System.out.println("Adams balance after deposit: " +
                           adamsBalance);

        System.out.println("Adams balance after withdrawal: " +
                           acct2.withdraw(480,1.50));

        acct3.withdraw(-100.00, 1.50); // invalid transaction

        acct1.addInterest();
    }
}
```

```

        acct2.addInterest();

        acct3.addInterest();

        System.out.println();

        System.out.println(acct1);

        System.out.println(acct2);

        System.out.println(acct3);
    }
}

```

OUTPUT

Adams balance after deposit: 575.25

Adams balance after withdrawal: 93.75

72354 Ted Murphy \$72.13

69713 Angelica Adams \$97.03

93757 Edward Demsey \$796.25

```

// LISTING 4.7

//*****

// Account.java

//

// Represents a bank account with basic services such as deposit

// and withdraw.

//*****

import java.text.NumberFormat;

public class Account

```



```
{

    private final double RATE = 0.035; // interest rate of 3.5%

    private String name;

    private long acctNumber;

    private double balance;

    //-----

    // Sets up this account with the specified owner, account number,
    // and initial balance.
    //-----

    public Account(String owner, long account, double initial)
    {

        name = owner;

        acctNumber = account;

        balance = initial;

    }

    //-----

    // Deposits the specified amount into this account and returns
    // the new balance. The balance is not modified if the deposit
    // amount is invalid.
    //-----

    public double deposit (double amount)
    {

        if (amount > 0)

            balance = balance + amount;

        return balance;

    }

}
```

```

}

//-----

// Withdraws the specified amount and fee from this account and
// returns the new balance. The balance is not modified if the
// withdraw amount is invalid or the balance is insufficient.
//-----

public double withdraw (double amount, double fee)
{
    if (amount+fee > 0 && amount+fee < balance)
        balance = balance - amount - fee;

    return balance;
}

//-----

// Adds interest to this account and returns the new balance.
//-----

public double addInterest()
{
    balance += (balance * RATE);

    return balance;
}

//-----

// Returns the current balance of this account.
//-----

public double getBalance()

```

```

{
    return balance;
}

//-----
// Returns a one-line description of this account as a string.
//-----

public String toString()
{
    NumberFormat fmt = NumberFormat.getCurrencyInstance();

    return (acctNumber + "\t" + name + "\t" + fmt.format(balance));
}
}

```

4.5 Static Class Members

We have encountered static methods in various contexts throughout previous chapters. All of the methods in the `Math` class, for example, are static. Recall that a static method is one that is invoked through the class name itself rather than through an instance of the class.

Methods are not the only members that can be declared static — variables can be static as well. Both are designated using the `static` modifier.

Deciding whether a given method or variable should be static is an important class design decision. Let's examine the implications of each more closely.

Static Variables

Up to this point we have worked with two categories of variables: local variables declared inside methods, and instance variables declared within a class but outside any method. The term instance variable reflects the fact that each instance of the class gets its own

independent copy of the variable, with its own memory space, enabling different objects to hold different values.

KEY CONCEPT

A static variable is shared among all instances of a class.

A static variable — also called a class variable — operates differently. Rather than giving each object its own copy, a single copy of the variable is shared across all instances of the class. Modifying that variable through one object changes it for every other object as well. A static variable is declared using the `static` modifier:

```
private static int count = 0;
```

Memory for a static variable is allocated the first time the class containing it is referenced in a running program. It is worth noting that local variables declared inside a method cannot be declared as `static`.

Constants, which are declared with the `final` modifier, are frequently declared `static` as well. Since a constant's value can never change, there is no reason to replicate it across every object — a single shared copy serves all instances equally well.

Static Methods

A static method can be invoked through the class name without instantiating any object of that class. The methods of the `Math` class are all static — for example, the following line invokes `sqrt` directly through the class name:

```
System.out.println ("Square root of 27:" + Math.sqrt(27));
```

The methods of `Math` perform calculations based entirely on their input parameters. There is no object state to maintain, so there is no meaningful reason to require the creation of an object just to access these services.

A method is made static by including the `static` modifier in its declaration. As we have seen in every program example so far, the `main` method is always declared static, precisely so the Java interpreter can invoke it without first instantiating an object from the class that contains it.

Because a static method does not execute in the context of any particular object, it cannot reference instance variables — those only exist within an object. Attempting to reference an instance variable from a static method will produce a compiler error. Static methods can, however, reference static variables, since those exist independently of any specific instance. As a consequence, all static methods — including `main` — are limited to accessing only static variables or locally declared variables.

The `SloganCounter` program in Listing 4.8 creates several `Slogan` objects, prints each one, and then retrieves the total count of slogans created by invoking a static method through the class name.

The `Slogan` class in Listing 4.9 includes a static variable `count` initialized to zero. The constructor increments `count` each time a new `Slogan` is instantiated, making it an ongoing tally of all objects ever created from the class. The `getCount` method is declared static so that it can be called through the class name without needing a `Slogan` object as a receiver. Because `getCount` only references the static variable `count`, the `static` declaration is valid. Had the method needed to access instance data, it could not have been declared static without triggering a compiler error.

It would have been legal to declare `getCount` without the `static` modifier, but then it would have had to be called through a specific `Slogan` instance rather than through the class name directly.

```
// LISTING 4.8

//*****

// SloganCounter.java

//

// Demonstrates the use of the static modifier.

//*****

public class SloganCounter
{
    //-----

    // Creates several Slogan objects and prints the number of
    // objects that were created.

    //-----

    public static void main(String[] args)
    {
        Slogan obj;

        obj = new Slogan ("Remember the Alamo.");
```

```

        System.out.println(obj);

        obj = new Slogan ("Don't Worry. Be Happy.");
        System.out.println(obj);

        obj = new Slogan ("Live Free or Die.");
        System.out.println(obj);

        obj = new Slogan ("Talk is Cheap.");
        System.out.println(obj);

        obj = new Slogan ("Write Once, Run Anywhere.");
        System.out.println(obj);

        System.out.println();

        System.out.println("Slogans created: " + Slogan.getCount());
    }
}

```

OUTPUT

Remember the Alamo.
 Don't Worry. Be Happy.
 Live Free or Die.
 Talk is Cheap.
 Write Once, Run Anywhere.
 Slogans created: 5

```

// LISTING 4.9

//*****

// Slogan.java

```

```
//  
  
// Represents a single slogan or motto.  
  
//*****  
public class Slogan  
{  
  
    private String phrase;  
  
    private static int count = 0;  
  
    //-----  
    // Constructor: Sets up the slogan and increments the number of  
    // instances created.  
    //-----  
    public Slogan (String str)  
    {  
        phrase = str;  
        count++;  
    }  
  
    //-----  
    // Returns this slogan as a string.  
    //-----  
    public String toString()  
    {  
        return phrase;  
    }  
  
    //-----
```

```
// Returns the number of instances of this class that have been
// created.
//-----
public static int getCount ()
{
    return count;
}
}
```

4.6 Class Relationships

The classes that make up a software system are not isolated from one another — they exist in various kinds of relationships. Three of the most prevalent are dependency, aggregation, and inheritance.

We have already encountered dependency relationships in many of our examples, where one class makes use of another. This section revisits dependency and also explores the interesting case in which a class depends on itself. We then turn to aggregation, in which objects of one class contain objects of another, forming a "has-a" relationship. Inheritance, which was introduced in Chapter 1 and creates an "is-a" relationship between classes.

Dependency

A dependency relationship exists when one class relies on another in some capacity — typically when the methods of one class invoke the methods of another. This is commonly described as a "uses" relationship.

More precisely, if class A depends on class B, then one or more methods in A call one or more methods in B. If the method being called is static, A references B by name alone. If the method is not static, A must hold a reference to a specific instance of B in order to invoke the method. How A acquires that reference — whether by creating the object itself or by receiving it as a method parameter — is an important design decision.

In general, it is good practice to minimize the number of dependencies between classes. The fewer the interdependencies, the more isolated and independent each class is, and the less likely that a change or error in one part of the system will propagate and affect others.

Dependencies Among Objects of the Same Class

In some cases, a class depends on itself — an object of one class interacts with another object of the very same class. This is accomplished by defining a method that accepts an object of its own class as a parameter.

The `concat` method of the `String` class is a familiar example. It is invoked through one `String` object and receives another as a parameter:

```
str3 = str1.concat(str2);
```

The calling object (`str1`) appends its own characters to those of the parameter object (`str2`), and the resulting new `String` is returned and stored in `str3`.

The `RationalTester` program in Listing 4.10 demonstrates a similar pattern. A rational number is any value expressible as a ratio of two integers — a fraction. The program creates two `RationalNumber` objects and performs a variety of arithmetic operations on them.

The `RationalNumber` class in Listing 4.11 represents a single rational number. Each arithmetic method — `add`, `subtract`, `multiply`, `divide` — treats the object through which it is invoked as the left operand and the `RationalNumber` passed as a parameter as the right operand.

The `isLike` method tests whether two rational numbers are equivalent. Although it is tempting to call this method `equals`, we will see in Chapter 5 that the `equals` method carries special significance in the context of inheritance and must be implemented in a specific way. To avoid confusion for now, we use the name `isLike` instead.

Note also that the `reduce` and `gcd` methods in `RationalNumber` are declared with `private` visibility. These methods exist only to support the publicly visible methods of the class — they are not intended to be called directly from outside a `RationalNumber` object. Declaring them `private` enforces this boundary cleanly.

```
// LISTING 4.10
//*****
// RationalTester.java
//
// Driver to exercise the use of multiple Rational objects.
```

```
//*****

public class RationalTester
{
    //-----

    // Creates some rational number objects and performs various
    // operations on them.

    //-----

    public static void main(String[] args)
    {
        RationalNumber r1 = new RationalNumber (6,8);
        RationalNumber r2 = new RationalNumber (1,3);
        RationalNumber r3, r4, r5, r6, r7;

        System.out.println("First rational number: " + r1);
        System.out.println("Second rational number: " + r2);

        if (r1.isLike(r2))
            System.out.println("r1 and r2 are equal.");
        else
            System.out.println("r1 and r2 are NOT equal.");

        r3 = r1.reciprocal();
        System.out.println("The reciprocal of r1 is: " + r3);

        r4 = r1.add(r2);
        r5 = r1.subtract(r2);
    }
}
```

```
        r6 = r1.multiply(r2);

        r7 = r1.divide(r2);

        System.out.println("r1 + r2: " + r4);

        System.out.println("r1 - r2: " + r5);

        System.out.println("r1 * r2: " + r6);

        System.out.println("r1 / r2: " + r7);

    }
}
```

OUTPUT

First rational number: 3/4
Second rational number: 1/3
r1 and r2 are NOT equal.
The reciprocal of r1 is: 4/3
r1 + r2: 13/12
r1 - r2: 5/12
r1 * r2: 1/4
r1 / r2: 9/4

Aggregation

Some objects in a system are themselves composed of other objects. A car, for example, is made up of an engine, a chassis, wheels, and numerous other components — each of which could itself be modeled as a separate object. In this sense, a car is an aggregate: an object composed, at least in part, of other objects. Aggregation describes a "has-a" relationship — a car has a chassis, a car has an engine.

In software terms, an aggregate object is any object that contains references to other objects as part of its instance data. The `Account` class is a subtle example: it contains a `String` object representing the account holder's name. We often think of strings as primitive values, but they are objects, which technically makes every `Account` an aggregate.

Aggregation is a specialized form of dependency. A class that is partly defined in terms of another class is necessarily dependent on that class, and the methods of the aggregate object typically call methods of the objects it contains.

KEY CONCEPT

An aggregate object is composed of other objects, forming a has-a relationship.

In UML, an aggregation relationship is represented by a line connecting the two classes, with an open diamond at the end nearest the containing class. It is worth noting that in most UML diagrams in this book, `String` is not shown as a separate class participating in an aggregation relationship, even when it technically qualifies. Because strings are so pervasive in Java programs, they are typically treated as if they were primitive types in UML diagrams to avoid unnecessary clutter.

```
// LISTING 4.11

//*****

// RationalNumber.java
//
// Represents one rational number with a numerator and denominator.
//*****

public class RationalNumber
{
    private int numerator, denominator;

    //-----
    // Constructor: Sets up the rational number by ensuring a nonzero
    // denominator and making only the numerator signed.
    //-----

    public RationalNumber (int numer, int denom)
    {
        if (denom == 0)
            denom = 1;

        // Make the numerator "store" the sign
        if (denom < 0)
        {
            numer = numerator * - 1;
        }
    }
}
```

```
        denom = denominator * -1;

    }

    numerator = numer;

    denominator = denom;

    reduce();

}

//-----
// Returns the numerator of this rational number.
//-----

public int getNumerator ()

{

    return numerator;

}

//-----
// Returns the denominator of this rational number.
//-----

public int getDenominator ()

{

    return denominator;

}

//-----
```

```

// Returns the reciprocal of this rational number.

//-----

public RationalNumber reciprocal ()
{
    return new RationalNumber(denominator,numerator);
}

//-----

// Adds this rational number to the one passed as a parameter.
// A common denominator is found by multiplying the individual
// denominators.
//-----

public RationalNumber add (RationalNumber op2)
{
    int commonDenominator = denominator * op2.getDenominator();
    int numerator1 = numerator * op2.getDenominator();
    int numerator2 = op2.getNumerator() * denominator;
    int sum = numerator1 + numerator2;

    return new RationalNumber (sum, commonDenominator);
}

//-----

// Subtracts the rational number passed as a parameter from this
// rational number.
//-----

public RationalNumber subtract (RationalNumber op2)

```

```

{

    int commonDenominator = denominator * op2.getDenominator();

    int numerator1 = numerator * op2.getDenominator();

    int numerator2 = op2.getNumerator() * denominator;

    int difference = numerator1 - numerator2;


    return new RationalNumber (difference, commonDenominator);

}


//-----
// Multiplies this rational number by the one passed as a
// parameter.
//-----

public RationalNumber multiply (RationalNumber op2)

{

    int numer = numerator * op2.getNumerator();

    int demon = denominator * op2.getDenominator();


    return new RationalNumber (numer, demon);

}


//-----
// Divides this rational number by the one passed as a parameter
// by multiplying by the reciprocal of the second rational.
//-----

public RationalNumber divide (RationalNumber op2)

{

```

```

        return multiply(op2.reciprocal());
    }

    //-----
    // Determines if this rational number is equal to the one passed
    // as a parameter. Assumes they are both reduced.
    //-----

    public boolean isLike (RationalNumber op2)
    {
        return (numerator == op2.getNumerator() &&
                denominator == op2.getDenominator());
    }

    //-----
    // Returns this rational number as a string.
    //-----

    public String toString ()
    {
        String result;

        if (numerator == 0)
            result = "0";
        else
            if (denominator == 1)
                result = numerator + "";
            else
                result = numerator + "/" + denominator;
    }

```



```

        return result;
    }

    //-----

    // Reduces this rational number by dividing both the numerator
    // and the denominator by their greatest common divisor.
    //-----

    private void reduce ()
    {
        if (numerator != 0)
        {
            int common = gcd (Math.abs(numerator), denominator);

            numerator = numerator / common;

            denominator = denominator / common;
        }
    }

    //-----

    // Computes and returns the greatest common divisor of the two
    // positive parameters. Uses Euclid's algorithm.
    //-----

    private int gcd (int num1, int num2)
    {
        while (num1 != num2)
        {
            if (num1 > num2)

```

```
        num1 = num1 - num2;

    else

        num2 = num2 - num1;

    return num1;
}
}
```

The `this` Reference

Before leaving the topic of class relationships, we should examine a special reference available in every Java program: the `this` reference. The word `this` is a reserved keyword in Java that allows an object to refer to itself. When a non-static method is executing, the `this` reference can be used inside that method to explicitly refer to the object through which the method was invoked.

For example, in a class called `ChessPiece` there might be a method called `move` that contains the following statement:

```
if (this.position == piece2.position)
    result = false;
```

Here, `this` makes explicit which position is being compared — it is the position of the object currently executing the method. If the method is invoked as:

```
bishop1.move();
```

then `this` refers to `bishop1`. If invoked as:

```
bishop2.move();
```

then `this` refers to `bishop2`.

One of the most common uses of `this` is inside constructors, where it helps distinguish formal parameters from instance variables that share the same names. The `Account` constructor in Listing 4.7 was written using different names for the parameters specifically to avoid this ambiguity:

```
public Account (String owner, long account, double initial)
{
```

```

    name = owner;
    acctNumber = account;
    balance = initial;
}

```

Using `this`, the same constructor can be written with parameter names that mirror the instance variable names more naturally:

```

public Account (String name, long acctNumber, double balance)
{
    this.name = name;
    this.acctNumber = acctNumber;
    this.balance = balance;
}

```

In this version, `this.name` refers to the instance variable, while `name` on the right-hand side refers to the formal parameter. This eliminates the need to invent different-but-equivalent names for parameters that conceptually represent the same things as the instance variables. This pattern arises frequently in constructors.

4.7 Method Design

Once classes have been identified and their primary responsibilities assigned, the focus shifts to designing the methods that will implement those behaviors. Some methods are simple and essentially write themselves. Others require more careful thought and deliberate planning.

KEY CONCEPT

A complex service provided by an object can be decomposed and make use of private support methods.

An algorithm is a systematic, step-by-step procedure for solving a problem. A recipe is an algorithm; so is a set of driving directions. Every method implements an algorithm that determines how it fulfills its purpose.

Algorithms are often sketched out using pseudocode — an informal blend of code-like structure and plain English. Pseudocode conveys the logical flow of a solution without being constrained by the exact syntax of any particular programming language or prematurely committed to specific programming constructs.

This section addresses two key aspects of method-level design: decomposing complex methods into simpler ones, and understanding the implications of passing objects as parameters.

```

// LISTING 4.12

// *****

```

```
// PigLatin.java

//

// Demonstrates the concept of method decomposition.

//*****

import java.util.Scanner;

public class PigLatin
{
    //-----

    // Reads sentences and translates them into Pig Latin.

    //-----

    public static void main(String[] args)
    {
        String sentence, result, another;

        Scanner scanner = new Scanner(System.in);

        do
        {
            System.out.println();

            System.out.println("Enter a sentence (no punctuation):");

            sentence = scanner.nextLine();

            System.out.println();

            result = PigLatinTranslator.translate(sentence);

            System.out.println("That sentence in Pig Latin is:");

            System.out.println(result);
        }
    }
}
```

```

        System.out.println();

        System.out.print("Translate another sentence (y/n)? ");

        another = scanner.nextLine();

    }

    while (another.equalsIgnoreCase("y"));

}

}

```

OUTPUT

Enter a sentence (no punctuation):
 Do you speak Pig Latin
 That sentence in Pig Latin is:
 oday ouyay eakspay igpay atinlay
 Translate another sentence (y/n)? y
 Enter a sentence (no punctuation):
 Play it again Sam
 That sentence in Pig Latin is:
 ayplay ityay againyay amsay
 Translate another sentence (y/n)? n

```

// LISTING 4.13
//*****
// PigLatinTranslator.java
//
// Represents a translator from English to Pig Latin. Demonstrates
// method decomposition.
//*****
import java.util.Scanner;

public class PigLatinTranslator
{
    //-----
    // Translates a sentence of words into Pig Latin.
    //-----
    public static String translate(String sentence)
    {
        String result = "";

        sentence = sentence.toLowerCase();

        Scanner scanner = new Scanner(sentence);
    }
}

```

```

        while (scanner.hasNext())
        {
            result += translateWord (scanner.next());
            result += " ";
        }

        return result;
    }

    //-----
    // Translates one word into Pig Latin. If the word begins with a
    // vowel, the suffix "yay" is appended to the word. Otherwise,
    // the first letter or two are moved to the end of the word,
    // and "ay" is appended.
    //-----

    private static String translateWord(String word)
    {
        String result = "";

        if (beginsWithVowel(word))
            result = word + "yay";
        else
            if (beginsWithBlend(word))
                result = word.substring(2) + word.substring(0, 2) +
"ay";
            else
                result = word.substring(1) + word.charAt(0) + "ay";

        return result;
    }

    //-----
    // Determines if the specified word begins with a vowel.
    //-----
    private static boolean beginsWithVowel(String word)
    {
        String vowels = "aeiou";

        char letter = word.charAt(0);

        return (vowels.indexOf(letter) != -1);
    }

    //-----
    // Determines if the specified word begins with a particular
    // two-character consonant blend.
    //-----
    private static boolean beginsWithBlend(String word)
    {
        return (word.startsWith("bl") || word.startsWith("sc") ||

```

```

        word.startsWith("br") || word.startsWith("sh") ||
        word.startsWith("ch") || word.startsWith("sk") ||
        word.startsWith("cl") || word.startsWith("sl") ||
        word.startsWith("cr") || word.startsWith("sn") ||
        word.startsWith("dr") || word.startsWith("sm") ||
        word.startsWith("dw") || word.startsWith("sp") ||
        word.startsWith("fl") || word.startsWith("sq") ||
        word.startsWith("fr") || word.startsWith("st") ||
        word.startsWith("gl") || word.startsWith("sw") ||
        word.startsWith("gr") || word.startsWith("th") ||
        word.startsWith("kl") || word.startsWith("tr") ||
        word.startsWith("ph") || word.startsWith("tw") ||
        word.startsWith("pl") || word.startsWith("wh") ||
        word.startsWith("pr") || word.startsWith("wr") );
    }
}

```

Method Decomposition

There are times when a service a class must provide is complex enough that implementing it in a single method produces code that is unwieldy and hard to follow. In such cases it is better to decompose the method into several smaller, more focused methods that collectively accomplish the goal. Let's examine a concrete example: translating English sentences into Pig Latin.

Pig Latin is an informal, made-up language. Each word of a sentence is transformed according to a set of rules. Generally, the initial consonant sound is moved to the end of the word and an "ay" sound is appended. The word "happy" becomes "appyhay" and "birthday" becomes "irthdaybay." Words that begin with a vowel simply have "yay" appended — "enough" becomes "enoughyay." When a word begins with a consonant blend such as "ch" or "st," the entire blend is moved to the end together before the "ay" is appended, so "grapefruit" becomes "apefruitgray."

The **PigLatin** program in Listing 4.12 reads one or more sentences and outputs their Pig Latin equivalents.

The core logic lives in the **PigLatinTranslator** class in Listing 4.13. It exposes a single public service: a static method called **translate** that accepts a sentence and returns its Pig Latin translation. No constructor is defined in this class because none is needed — all functionality is delivered through the static method.

Translating an entire sentence at once is not a trivial task. Written as one large method, the code would be long, tangled, and difficult to reason about. The better approach — and the one used here — is to break **translate** into a collection of smaller, focused support methods.

The `translate` method uses a `Scanner` to break the input sentence into individual words, using the default whitespace delimiters. Each word is handed off to a private method called `translateWord`. Translating a single word is itself nontrivial, so `translateWord` relies on two additional private methods: `beginsWithVowel` and `beginsWithBlend`.

The `beginsWithVowel` method returns a `boolean` indicating whether a word's first character is a vowel. Rather than testing each vowel separately with a chain of comparisons, it constructs a string containing all vowels and uses the `indexOf` method to check whether the first character of the word appears in it. A return value of `-1` from `indexOf` means the character is not found.

The `beginsWithBlend` method also returns a `boolean`. Its body consists of a single `return` statement containing a large compound expression that calls `startsWith` on the word for each recognized consonant blend. If any of those calls returns `true`, the whole expression — and therefore the method — returns `true`.

All three support methods — `translateWord`, `beginsWithVowel`, and `beginsWithBlend` — are declared `private`. They are not intended to be called from outside the class; their sole purpose is to assist the public `translate` method. If code in the `PigLatin` class were to attempt calling `translateWord` directly, the compiler would reject it immediately.

Whenever a method grows large or complex, breaking it into smaller pieces is the right instinct. However, this must not be considered a substitute for the more fundamental step of identifying the right classes and objects at the system level. In object-oriented design, decomposing responsibilities across well-defined objects always takes precedence over decomposing within the methods of a single class.

Method Parameters Revisited

A second important design consideration at the method level involves how parameters are passed. In Java, all parameters are passed by value. When a method is invoked, the current value of each actual parameter is copied into the corresponding formal parameter. This is conceptually equivalent to an assignment statement that assigns the actual parameter's value to the formal parameter at the point of the call.

One important consequence of this is that changes made to a formal parameter inside a method have no effect on the actual parameter in the calling code. The formal parameter holds an independent copy, and once the method returns, the actual parameter retains whatever value it had before the call.

KEY CONCEPT

When an object is passed to a method, the actual and formal parameters become aliases.

However, when an object is passed as a parameter, the situation is subtly different. What gets copied is not the object itself, but the reference — the memory address — of the object. As a result, the formal parameter and the actual parameter both point to the same object in memory; they become aliases of each other. Any change made to the state of the object through the formal parameter inside the method is a change to the same object that the actual parameter refers to outside the method. That change persists after the method returns.

There is one important nuance: if, inside the method, the formal parameter is reassigned to point to an entirely new object, that change does not propagate back to the caller. The actual parameter continues to reference the original object, unaffected. Only modifications made to the object's state — through method calls or direct field access — are visible to the caller after the method returns.

The `ParameterTester` program in Listing 4.14 makes these distinctions concrete. Its `main` method calls `changeValues` on a `ParameterModifier` object, passing a primitive integer and two `Num` objects (each of which simply stores an integer value). The `ParameterModifier` class appears in Listing 4.15, and the `Num` class is shown in Listing 4.16.

Inside `changeValues`, three modifications are made: the primitive integer parameter is assigned a new value, the internal value of the first `Num` object is changed via its `setValue` method, and the second `Num` formal parameter is reassigned to a newly created `Num` object. All three changes are visible in the output printed at the end of `changeValues`.

After control returns to `main`, however, the picture changes significantly. The primitive integer has reverted to its original value — the change was made to a local copy. The second `Num` parameter also shows its original value, because reassigning the formal parameter to a new object had no effect on what the actual parameter references. The only lasting change is the one made to the internal state of the first `Num` object via `setValue` — because both the formal and actual parameters referred to the same object in memory, that modification persisted.

4.8 Method Overloading

As we have established, invoking a method transfers control to the code that defines it. Once the method has finished executing, control returns to the call site and the program continues from there.

In most situations, the method name alone is sufficient to identify which method is being invoked. In Java, however — as in other object-oriented languages — multiple methods can share the same name, provided they differ in their parameter lists. This technique is called method overloading, and it is particularly useful when conceptually similar operations need to be performed on different types of data.

KEY CONCEPT

The versions of an overloaded method are distinguished by the number, type, and order of their parameters.

The compiler must still be able to resolve each invocation to a single unambiguous method definition. When two or more methods share a name, the compiler uses the number of parameters, their types, and their order to determine which version is being called. As long as these characteristics differ, methods with the same name can coexist within the same class.

For example, we could define two methods both named `sum`:

```
public int sum (int num1, int num2)

{

    return num1 + num2;

}
```

and:

```
public int sum (int num1, int num2, int num3)

{

    return num1 + num2 + num3;

}
```

When an invocation such as the following appears:

```
sum (25, 69, 13);
```

the compiler counts the parameters and routes the call to the three-parameter version.

The combination of a method's name and its parameter list — specifically the number, types, and order of its parameters — is called the method's signature. The compiler relies on the full signature to bind each invocation to the correct definition.

If two methods are declared with identical signatures, the compiler will flag the duplication as an error and refuse to compile the program. Every invocation must map to exactly one method, without ambiguity.

One important point: the return type of a method is not part of its signature. Two overloaded versions of a method cannot differ solely by their return types. Because a caller may choose to ignore the return value entirely, the compiler would have no way to determine which version was intended in such cases.

The `println` method is a familiar and widely used example of overloading. It is defined in multiple versions, each accepting a different parameter type:

- `println (String s)`
- `println (int i)`
- `println (double d)`
- `println (char c)`
- `println (boolean b)`

The following two statements each invoke a different version of `println`:

```
System.out.println ("Number of students: ");
```

```
System.out.println (count);
```

The first calls the version that accepts a `String`. The second — assuming `count` is an integer — calls the version that accepts an `int`.

When we write a single `println` that combines both, such as:

```
System.out.println ("Number of students: " + count);
```

the `+` operator performs string concatenation: the integer value of `count` is first converted to a string, the two strings are joined, and the single-string version of `println` is invoked with the result.

Constructors can also be overloaded — and frequently are. Providing multiple constructor versions gives clients different ways to initialize an object at the point of creation, with varying amounts of initial information.

```
// LISTING 4.14

//*****

// ParameterTester.java
//
// Demonstrates the effects of passing various types of parameters.
//*****

public class ParameterTester
{

    //-----
```

```

// Sets up three variables (one primitive and two objects) to
// serve as actual parameters to the changeValues method. Prints
// their values before and after calling the method.
//-----

public static void main(String[] args)
{
    ParameterModifier modifier = new ParameterModifier();

    int a1 = 111;

    Num a2 = new Num(222);

    Num a3 = new Num(333);

    System.out.println("Before calling changeValues:");

    System.out.println("a1\ta2\ta3");

    System.out.println(a1 + "\t" + a2 + "\t" + a3 + "\n");

    modifier.changeValues(a1, a2, a3);

    System.out.println("After calling changeValues:");

    System.out.println("a1\ta2\ta3");

    System.out.println(a1 + "\t" + a2 + "\t" + a3 + "\n");

}
}

```

After changing the values:

f1 f2 f3

999 888 777

After calling changeValues:

a1 a2 a3

```
// LISTING 4.15

//*****

// ParameterModifier.java Java Foundations
//
// Demonstrates the effects of changing parameter values.
//*****

public class ParameterModifier
{
    //-----

    // Modifies the parameters, printing their values before and
    // after making the changes.
    //-----

    public void changeValues (int f1, Num f2, Num f3)
    {
        System.out.println("Before changing the values:");
        System.out.println("f1\tf2\tf3");
        System.out.println(f1 + "\t" + f2 + "\t" + f3 + "\n");

        f1 = 999;
        f2.setValue(888);
        f3 = new Num(777);

        System.out.println("After changing the values:");
        System.out.println("f1\tf2\tf3");
        System.out.println(f1 + "\t" + f2 + "\t" + f3 + "\n");
    }
}
```

```
}  
  
}
```

```
// LISTING 4.16  
  
//*****  
  
// Num.java Java Foundations  
  
//  
  
// Represents a single integer as an object.  
  
//*****  
  
public class Num  
{  
  
    private int value;  
  
  
    //-----  
    // Sets up the new Num object, storing an initial value.  
    //-----  
  
    public Num(int update)  
    {  
  
        value = update;  
  
    }  
  
  
    //-----  
    // Sets the stored value to the newly specified value.  
    //-----  
  
    public void setValue(int update)
```

```
{  
  
    value = update;  
  
}  
  
//-----  
  
// Returns the stored integer value as a string.  
  
//-----  
  
public String toString()  
  
{  
  
    return value + "  
  
}  
}
```

4.9 Testing

As programs grow in size and complexity, ensuring their correctness and reliability becomes increasingly challenging. Before pressing further into programming details, let's step back and examine what the testing process actually involves.

The word "testing" covers a broad range of activities in software development. In the traditional sense, testing means running a completed program with various inputs and observing whether problems arise. But in the broader sense, testing encompasses any evaluation — by a person or a tool — that assesses the quality of the software at any stage of its development. Meaningful testing should begin long before a single line of code is written.

The fundamental goal of testing is to find errors. By discovering and correcting them, we improve the quality and reliability of the program. If errors are not found during development, they will almost certainly be found later — by users — at a far greater cost in time, money, and reputation. Investing effort in uncovering problems early is nearly always worthwhile.

Running a program with a specific set of inputs and observing correct output establishes only that the program works for that particular scenario. As the collection of successful test cases grows, our confidence in the program increases — but we can never be fully certain that all errors have been eliminated. There may always be an undiscovered edge case

lurking somewhere. This is why testing must be as thorough and varied as possible, with carefully designed test cases rather than ad hoc runs.

Proving a program correct in the mathematical sense is possible, but the complexity of doing so for large systems is prohibitive, and even proofs are susceptible to errors. In practice, testing remains our primary means of assessing software quality.

After an error is found and corrected, all previously passing tests should be run again to confirm that the fix has not introduced new problems elsewhere. This practice is known as regression testing.

KEY CONCEPT

Testing a program can never guarantee the absence of errors.

Reviews

One technique for evaluating design or code before execution is the review — a structured meeting in which several people carefully examine a design document or section of code. The act of presenting your work to others encourages more careful preparation and opens the work to external scrutiny. Participants discuss both strengths and weaknesses, and compile a list of issues requiring attention. The goal of a review is to surface problems, not to solve them on the spot — that work happens separately and typically takes considerably more time.

A design review assesses whether the stated requirements are adequately addressed and whether the decomposition of the system into classes and objects is sound. A code review examines how faithfully the implementation reflects the design and the requirements, and identifies any specific issues that would cause the code to fail in its responsibilities.

Reviews in which participants step sequentially through a document, evaluating each section in order, are sometimes referred to as walkthroughs.

Defect Testing

Because the aim of testing is to find errors, testing is often called defect testing. A good test is one that succeeds in uncovering a deficiency in the program. This might seem counterintuitive, since we ultimately want the program to work — but the point is that errors almost certainly exist. Proactively finding and fixing them is far preferable to allowing users to discover them in production.

KEY CONCEPT

A good test is one that uncovers an error.

A test case is a defined set of inputs, conditions, and expected outputs. Test cases should be documented in enough detail that they can be repeated reliably in the future. Developers

often assemble a complete test suite — a curated collection of test cases that collectively exercise the different aspects and behaviors of the system.

KEY CONCEPT

It is not feasible to exhaustively test a program for all possible input and user actions.

Because the space of possible inputs is typically enormous, attempting to test every conceivable scenario is not practical. Nor is it usually necessary — two very similar test cases may exercise the same program paths and therefore provide no additional insight. A more productive strategy is to design each test case to exercise the program in some genuinely new way.

Two principal approaches to defect testing guide how test cases are selected: black-box testing and white-box testing.

Black-box testing treats the thing being tested as an opaque unit — its internal structure is irrelevant. Test cases are derived entirely from the specified inputs and expected outputs, without reference to how the code is implemented. Applied at the program level, inputs are user-provided data and actions; applied at the class level, it focuses on the public interface — what parameters go in and what results come out. Black-box tests are typically derived from the system's requirements or from the stated contract of a method.

Input data for black-box tests is often selected by identifying equivalence categories — groups of inputs expected to produce similar behavior. If a method works correctly for one value within a category, it is likely to work for all values in that category. For example, inputs to a square root method can be split into two categories: nonnegative integers (valid) and negative integers (invalid). One test case from each category is usually sufficient to cover the category's interior.

The boundaries between equivalence categories deserve special attention, since "off-by-one" errors are extremely common in programming. For any integer boundary, a comprehensive test suite should include at minimum the boundary value itself, one value below it, and one value above it.

As an illustration, consider a method that validates whether an integer falls between 0 and 99 inclusive. Three equivalence categories exist: values below 0, values from 0 to 99, and values above 99. An appropriate set of black-box test cases might be: -500, -1, 0, 1, 50, 98, 99, 100, and 500.

White-box testing — also known as glass-box testing — focuses on the internal structure and logic of the code rather than its external behavior. The goal is to ensure that every executable path through the program is traversed by at least one test case. This is sometimes called statement coverage.

Control flow through code is determined by conditional expressions in constructs like `if` statements and loops. To achieve full path coverage, test inputs must be chosen so that each conditional expression evaluates to both `true` and `false` across the test suite. This

guarantees that both branches of every `if` statement are exercised at least once. Similar reasoning applies to loop conditions and other flow-control constructs.

For both black-box and white-box testing, the expected output for each test case must be determined before the test is run. Deciding what the result should be only after seeing the actual output introduces the risk of rationalization — it is far too easy to convince yourself that an incorrect result is acceptable if you have no prior expectation to compare it against.

Unit Testing

Unit testing targets individual methods one at a time. For each method the developer has written, a corresponding test case is created to verify that the method behaves correctly in isolation. The goal is to establish the correctness of each unit of functionality before integrating it with the rest of the system. Test cases are typically gathered together and executed as a batch, with all results inspected collectively. These tests can also be run repeatedly as the codebase evolves, supporting regression testing by revealing when a change breaks something that previously worked.

Integration Testing

Integration testing follows unit testing and examines what happens when previously tested modules are brought together. A method that functions perfectly in isolation may interact unexpectedly with other methods or classes when combined. Integration testing identifies defects that emerge from those interactions. As with unit testing, regression testing can be applied here as well — re-running integration tests after each change helps detect newly introduced problems and confirms that earlier fixes remain intact. The goal of integration testing is typically to verify the correctness of larger, composite components of the system.

System Testing

System testing examines the complete, assembled software system and assesses how well its actual behavior matches its stated requirements. Public alpha and beta testing of applications and operating systems are familiar examples of system testing conducted prior to a product's formal release. Software development companies use these programs to expand the population of users exercising the product and to broaden the range of hardware configurations it is tested on.

Test-Driven Development

Ideally, test cases are developed alongside — or even ahead of — the source code they are intended to validate. Many developers have adopted a practice in which test cases are written first, and only enough implementation code is then written to make each test pass. This methodology is known as test-driven development.

Test-driven development demands that the developer test their code frequently throughout the development process. The sequence of activities generally looks like this:

1. Write a test case targeting a method that has not yet been implemented.
2. Run all existing test cases and confirm that every one passes except the newly added one.
3. Implement just enough code in the targeted method to make the new test case pass.
4. Run all test cases again and verify that every one — including the newly added test — now passes.
5. Review and clean up the code to eliminate any redundancy or inefficiency introduced during the implementation step. This cleanup phase is called refactoring.
6. Return to step 1 and repeat the cycle.

Test-driven development is gaining widespread adoption in professional software engineering. It does require a shift in habit — many developers find it uncomfortable to write tests before writing the code those tests are meant to exercise — but the discipline pays off in cleaner designs and fewer defects.

KEY CONCEPT

In test-driven development, test cases are developed for code before the code is written.

4.10 Debugging

As you grow as a programmer, one of the most important skills you will develop is debugging — the ability to locate and eliminate errors in your programs. A run-time error manifests itself as an abnormal program termination triggered by a condition the code did not anticipate. A logic error is subtler: the program runs to completion but produces results that do not match what was expected.

KEY CONCEPT

Debugging is the act of locating and correcting run-time and logic errors from your programs.

Rigorous testing aims to surface both types of errors, but in practice some will inevitably slip through into the final product. Once an error is recognized, the next step is to identify exactly where in the code it originates. For a divide-by-zero error, that means pinpointing the precise line of code where it occurs and examining the values of the variables involved in the operation. The same investigative process applies to logic errors.

Regardless of the specific error, it is often extremely valuable to observe the internal state of the program while it executes — the values of variables, the state of objects, and the path the program actually followed. A debugger is the tool designed for exactly this purpose.

KEY CONCEPT

A debugger is a software program that permits developers to observe the execution of a program.

Simple Debugging with `print` Statements

The most elementary debugging technique requires no special tools at all: placing `print` and `println` statements at strategic locations throughout the code to output information about the program's current state. These statements can display the value of specific variables or the string representation of objects at key moments during execution — useful for observing how an object's state evolves over time.

Beyond inspecting data values, print statements can also shed light on the path of execution. A developer uncertain about how far the program progressed before crashing might scatter "It got here." style messages throughout the code to trace the exact sequence of statements that were reached.

Print statements are also particularly useful around method calls. Printing the values of each parameter immediately after a method begins reveals precisely how the method was invoked — a technique that proves especially valuable when debugging recursive methods, covered in the next chapter. Similarly, printing the value of a variable just before it is returned can confirm that the correct result is being sent back to the caller.

Debugging Concepts

Print-statement debugging has its limits. For pinpointing specific values or tracking the path of execution at a coarse level, it is adequate. But for deeper investigation, a dedicated debugger offers significantly more power and precision. A debugger allows the developer to control program execution interactively and inspect the program's internal state in ways that print statements simply cannot provide.

A debugger supports the following core operations:

Setting breakpoints. The developer can flag specific lines of source code. When the program reaches a flagged line during execution, it pauses automatically, giving the developer the opportunity to inspect the program's current state.

Inspecting variables and objects. Once execution has paused at a breakpoint, the debugger provides a view of the current values of any variable or the complete state of any object, displayed within the debugger interface.

Stepping into or over a method. When execution is paused at a line that calls a method, the developer can choose to step into the method — entering its body and continuing the debug session inside — or to step over it, allowing the method to execute normally without examining its internals. Stepping over is appropriate for well-tested library methods like `System.out.println`, whose behavior we trust and cannot modify anyway.

Executing one statement at a time. After pausing at a breakpoint, the developer can advance execution one statement at a time, a process called single-stepping. This provides the most granular possible view of how the program executes, and offers the opportunity to inspect variable values after each individual statement.

Continuing execution. At any pause point, the developer can also choose to resume normal execution. The program will then run without interruption until it completes, encounters another breakpoint, or terminates due to a run-time error.

Debuggers offer many additional features beyond these fundamentals, but the capabilities listed above represent the essential toolkit. Any debugger worth using will support at minimum these operations.

Summary of Key Concepts

- The nouns in a problem description may indicate some of the classes and objects needed in a program.
- The heart of object-oriented programming is defining classes that represent objects with well-defined state and behavior.
- The scope of a variable, which determines where it can be referenced, depends on where it is declared.
- A UML class diagram helps us visualize the contents of and relationships among the classes of a program.
- An object should be encapsulated, guarding its data from inappropriate access.
- Instance variables should be declared with `private` visibility to promote encapsulation.
- Most objects contain accessor and mutator methods to allow the client to manage data in a controlled manner.
- The way a class represents an object's state should be independent of how that object is used.
- The value returned from a method must be consistent with the return type specified in the method header.
- When a method is called, the actual parameters are copied into the formal parameters.
- A variable declared in a method is local to that method and cannot be used outside of it.
- A constructor cannot have any return type, even `void`.
- A static variable is shared among all instances of a class.
- An aggregate object is composed of other objects, forming a has-a relationship.
- A complex service provided by an object can be decomposed to make use of private support methods.
- When an object is passed to a method, the actual and formal parameters become aliases.
- The versions of an overloaded method are distinguished by the number, type, and order

of their parameters.

- Testing a program can never guarantee the absence of errors.
- A good test is one that uncovers an error.
- It is not feasible to exhaustively test a program for all possible input and user actions.
- In test-driven development, test cases are developed for code before the code is written.
- Debugging is the act of locating and correcting run-time and logic errors from your programs.
- A debugger is a software program that permits developers to observe the execution of a program.